Busca de alunos por matrícula: tabela hash ou árvore AVL?

Gabriel Ferreira Pereira — Ciência dos Dados — UFMS
Estrutura de Dados — 2º período

Uma universidade precisa manter um cadastro de alunos para consultar informações como nome, curso e situação da matrícula. Considere um sistema com aproximadamente 50 mil registros, no qual cada aluno possui um número de matrícula único. A operação mais frequente é informar uma matrícula completa e localizar o cadastro correspondente. Também são realizadas inserções de novos alunos, atualizações de informações e exclusões de registros. Nesse contexto, não é necessário apresentar os alunos em ordem de matrícula nem pesquisar intervalos de números com frequência.

Esse problema pode ser resolvido tanto por uma tabela hash quanto por uma árvore binária de busca ou uma árvore AVL. Nas três estruturas, a matrícula funciona como chave e permite localizar o registro do aluno. A diferença está na maneira de organizar as chaves e no trabalho necessário para executar cada operação. Para comparar as alternativas, consideramos o custo das operações na memória, supondo que comparar ou calcular o hash de uma matrícula tenha custo constante. A leitura de arquivos e a comunicação pela rede ficam fora dessa análise.

Na tabela hash, uma função transforma a matrícula em um índice que indica onde o registro deve ser procurado. Matrículas diferentes podem produzir o mesmo índice, situação chamada de colisão. Uma forma de resolvê-la é manter, em cada posição da tabela, uma lista dos registros que chegaram àquele índice. Durante a busca, a função identifica a posição e as chaves armazenadas nela são comparadas até encontrar o aluno. Com uma função adequada e o fator de carga controlado, busca e exclusão apresentam tempo esperado O(1). A inserção também tem custo esperado constante, amortizado quando consideramos os redimensionamentos.

Para o sistema descrito, a tabela hash é a melhor opção, pois a consulta exata por matrícula é muito mais frequente que qualquer operação que dependa da ordenação. Com uma distribuição adequada das chaves, o trabalho médio de localização permanece próximo de constante mesmo quando o cadastro cresce. Essa escolha exige acompanhar o fator de carga, que relaciona a quantidade de registros ao tamanho da tabela, e aumentar sua capacidade quando necessário. O objetivo não é eliminar todas as colisões, mas evitar que elas se concentrem e prejudiquem o desempenho.

Comparação das estruturas e escolha da solução

Em uma árvore binária de busca, as matrículas menores que a chave de um nó ficam à esquerda e as maiores ficam à direita. Para localizar um aluno, comparamos a matrícula procurada com a do nó atual e seguimos pelo lado correspondente. O custo de busca, inserção e exclusão depende da altura da árvore: O(h). Se a árvore estiver equilibrada, sua altura será O(log n). Porém, uma árvore binária de busca comum pode ficar degenerada, por exemplo, quando recebe matrículas em ordem crescente, formando uma sequência de nós com altura proporcional à quantidade de registros. Nesse caso, as operações podem chegar a O(n).

A árvore AVL resolve esse problema ao manter o equilíbrio por meio de rotações após inserções e exclusões, quando necessárias. Dessa forma, sua altura permanece O(log n), garantindo esse limite para busca, inserção e exclusão no pior caso. Em um cadastro de 50 mil alunos, isso representa percorrer uma quantidade de níveis muito menor que o total de registros. Sua principal vantagem em relação à tabela hash é oferecer um limite de tempo mais previsível, sem depender da distribuição produzida por uma função de hashing.

A tabela hash, por outro lado, pode apresentar operações O(n) no pior caso, caso muitas chaves sejam direcionadas à mesma posição na implementação com listas. Além disso, aumentar a tabela exige redistribuir os registros, processo com custo O(n) que pode causar uma demora pontual. Embora essa despesa possa ser diluída entre várias inserções, ela deve ser considerada em sistemas sensíveis a atrasos. Matrículas sequenciais ou pares não tornam a tabela necessariamente ruim: o resultado depende da função escolhida, da capacidade da tabela e do tratamento de colisões.

A AVL também apresenta desvantagens. Cada nó precisa guardar referências para outros nós e informações de balanceamento, enquanto inserções e exclusões podem exigir rotações. Sua implementação é mais trabalhosa e, para consultas exatas, O(log n) tende a exigir mais etapas que o O(1) esperado de uma tabela hash bem configurada. Em contrapartida, a AVL mantém as chaves ordenadas: permite listar todos os registros em O(n) e consultar um intervalo em O(log n + k), sendo k a quantidade de resultados. Na tabela hash, produzir uma lista ordenada geralmente exige coletar os registros e ordená-los, com custo O(n log n) usando um algoritmo de comparação eficiente.

Portanto, escolheria a tabela hash para esse cadastro, devido ao predomínio das buscas por uma matrícula completa. Utilizaria uma função que distribuísse adequadamente as chaves e uma política de redimensionamento para controlar as colisões. Se os requisitos mudassem e passassem a incluir consultas frequentes por intervalos de matrícula, listagens ordenadas ou garantias de desempenho no pior caso, a árvore AVL seria mais adequada. A melhor estrutura depende das operações exigidas pelo sistema e das características do conjunto de dados.